бюджетное профессиональное образовательное учреждение
Вологодской области
«Череповецкий лесомеханический техникум им. В.П. Чкалова»

Специальность 09.02.07 Информационные системы и программирование

ДИПЛОМНЫЙ ПРОЕКТ

РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА АНАЛИЗА И ГЕНЕРАЦИИ СЕТЕВОГО ТРАФИКА

СТУДЕНТ: 4 курса группы ИС-42
Неклюдов Никита Сергеевич

НАУЧНЫЙ РУКОВОДИТЕЛЬ: _____
преподаватель
Табунов П.А.

РЕЦЕНЗЕНТ: _____

ДОПУЩЕНО К ЗАЩИТЕ: «___» _____ 2026г.

ДАТА ЗАЩИТЫ: «___» _____ 2026г.

ЗАВ. ОТДЕЛЕНИЕМ:

ОЦЕНКА ЗАЩИТЫ: _____

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТРЕБОВАНИЙ 7
1.1 Характеристика предметной области: сетевой трафик и протоколы 7
1.2 Обзор существующих средств анализа и генерации 8
1.3 Постановка задачи и требования к функционалу 11
1.3.1. Требования к анализу и мониторингу трафика 11
1.3.2. Требования к генерации сетевых пакетов 11
1.3.3. Требования к фильтрации и хранению данных 12
1.4 Требования к безопасности и этике использования 13
1.4.1 Предотвращение несанкционированного использования (Misuse) 13
1.4.2 Этические аспекты и предотвращение сетевых атак 13
1.5 Выбор средств разработки 14
2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА 16
2.1 Архитектура приложения и модульная структура 16
2.2 Проектирование классовой модели 17
2.3 Моделирование процессов обработки пакетов 20
2.4 Алгоритмы генерации и анализа трафика 21
2.5 Структура данных и хранение результатов 23
2.6 Проектирование интерфейса взаимодействия 24
3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ 27
3.1 Программная реализация классов и методов 27
3.2 Работа с сетевыми сокетами и сырыми пакетами 27
3.3 Реализация механизмов фильтрации и статистики 27
3.4 Тестирование функциональности 27
3.5 Оценка производительности и нагрузки на систему 27
4. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ 28
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 29
ПРИЛОЖЕНИЕ А 30
ПРИЛОЖЕНИЕ Б 35
ПРИЛОЖЕНИЕ В 36

ВВЕДЕНИЕ

На сегодняшний день функционирование любой современной организации невозможно без использования локальных и глобальных вычислительных сетей. С развитием технологий «интернета вещей» (IoT), облачных вычислений и распределенных систем, объемы передаваемого трафика растут в геометрической прогрессии. В связи с этим, вопросы обеспечения стабильности, производительности и безопасности сетевой инфраструктуры выходят на первый план.

Мониторинг и глубокий анализ сетевого трафика являются базовыми процессами в работе системного администратора и специалиста по информационной безопасности. Инструментарий для выполнения этих задач должен быть не только мощным, но и гибким. Существующие на рынке программные решения часто обладают рядом недостатков для специфических задач. Например, профессиональные анализаторы уровня Wireshark обладают избыточным функционалом, который требует значительных системных ресурсов и длительного освоения, а консольные утилиты вроде tcpdump не всегда обеспечивают необходимую наглядность и интерактивность при генерации специфических сетевых пакетов.

Особое значение приобретает возможность создания собственных инструментов на базе высокоуровневых языков программирования. Выбор языка Python и библиотеки Scapy для реализации дипломного проекта обусловлен их способностью работать с «сырыми» сокетами (Raw Sockets), что позволяет формировать пакеты произвольной структуры. Это критически важно не только для пассивного анализа, но и для активного тестирования сетевого оборудования на устойчивость к различным типам нагрузок. Таким образом, разработка программного комплекса, совмещающего в себе функции мониторинга и гибкой генерации трафика, является актуальной задачей, отвечающей современным требованиям к инструментам сетевого администрирования.

Объект исследования: процессы передачи и обработки пакетов данных в вычислительных сетях на базе стека протоколов TCP/IP.

Предмет исследования: программные алгоритмы и методы перехвата, декодирования и формирования пакетов данных с использованием специализированных библиотек анализа трафика.

Цель работы: проектирование и программная реализация многофункционального комплекса для мониторинга и формирования сетевого трафика, предоставляющего пользователю инструменты для детального разбора пакетов и тестирования сетевых соединений.

Для достижения поставленной цели в рамках дипломного проекта необходимо решить следующие задачи:

1. Выполнить подробный анализ предметной области, изучив теоретические аспекты функционирования модели OSI и стека TCP/IP.
2. Провести сравнительный обзор существующих программных средств анализа трафика, выделив их сильные и слабые стороны для обоснования необходимости разработки собственного решения.
3. Обосновать выбор технологического стека (Python, Scapy, Npcap) и среды разработки PyCharm как наиболее эффективных инструментов для решения поставленных задач.
4. Разработать детальное техническое задание на создание программного продукта в соответствии с требованиями ГОСТ 19.201-78.
5. Спроектировать архитектуру программного комплекса, подготовить полный пакет UML-диаграмм (классов, последовательности, деятельности), описывающих внутреннюю логику работы приложения.
6. Реализовать программный код модулей захвата, анализа и генерации сетевых пакетов.
7. Провести серию испытаний разработанного средства в контролируемой сетевой среде, выполнить отладку и оценку соответствия функционала заявленным требованиям.

8. Оформить пояснительную записку и подготовить демонстрационные материалы для защиты проекта.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТРЕБОВАНИЙ

1.1 Характеристика предметной области: сетевой трафик и протоколы

Функционирование современных информационных систем неразрывно связано с обменом данными через компьютерные сети. Под сетевым трафиком понимается совокупность данных, передаваемых по каналам связи в определенный промежуток времени. Понимание природы этого трафика, его структуры и механизмов передачи является фундаментом для разработки средств мониторинга и генерации пакетов.

В основе сетевого взаимодействия лежит эталонная модель взаимодействия открытых систем (OSI), которая разделяет процесс передачи данных на семь логических уровней. Для разработки программного комплекса мониторинга наиболее критичными являются первые четыре уровня, так как именно на них происходит формирование структуры кадров и пакетов, с которыми работает библиотека Scapy:

1. Физический уровень (Physical): определяет физические характеристики каналов связи. На этом уровне наш программный комплекс взаимодействует с сетевым адаптером, переведенным в «смешанный режим» (promiscuous mode), что позволяет захватывать все пакеты, проходящие через сегмент сети, а не только адресованные конкретному узлу.
2. Канальный уровень (Data Link): здесь данные оформляются в кадры (frames). Основным протоколом является Ethernet. В рамках дипломного проектирования важно учитывать структуру Ethernet-кадра, включающую MAC-адреса отправителя и получателя, а также поле типа протокола верхнего уровня.
3. Сетевой уровень (Network): отвечает за маршрутизацию данных. Ключевым протоколом здесь выступает IP (Internet Protocol). Для анализатора трафика принципиально важно уметь разбирать IP-

заголовки, извлекая информацию об адресации, времени жизни пакета (TTL) и контрольных суммах.

4. Транспортный уровень (Transport): обеспечивает доставку данных между процессами. Основные протоколы — TCP (с установлением соединения) и UDP (без установления соединения). Разрабатываемое программное средство должно корректно идентифицировать порты приложений и анализировать флаги TCP-пакетов, что необходимо для диагностики состояния сетевых служб.

Стек протоколов TCP/IP, являющийся практической реализацией модели OSI, накладывает свои особенности на структуру трафика. Каждый пакет представляет собой многослойную структуру, где данные одного уровня инкапсулируются в заголовки другого. Процесс анализа трафика (сниффинг) по сути является процессом деинкапсуляции — последовательного «снятия» заголовков для изучения содержимого на каждом этапе.

Генерация трафика, в свою очередь, представляет собой обратный процесс. В дипломном проекте упор делается на создание инструмента, который позволит программно формировать пакеты, начиная с нижних уровней. Это дает возможность симулировать различные сетевые ситуации: от стандартных запросов до специфических нагрузочных тестов. Использование языка Python позволяет автоматизировать этот процесс, делая его более гибким по сравнению с жестко заданными шаблонами в готовых утилитах.

Таким образом, предметная область охватывает широкий спектр технологий передачи данных. Глубокое понимание структуры заголовков и логики инкапсуляции протоколов TCP/IP является необходимым условием для реализации алгоритмов захвата и формирования трафика, которые будут подробно рассмотрены в последующих разделах работы.

1.2 Обзор существующих средств анализа и генерации

Для обоснования необходимости разработки собственного программного комплекса необходимо провести критический анализ наиболее востребованных инструментов, представленных на рынке программного обеспечения для сетевого администрирования. Основными критериями оценки выступают: функциональность, доступность, удобство интерфейса и возможность кастомной генерации пакетов.

В качестве объектов сравнения выбраны три наиболее репрезентативных средства: Wireshark, tcpdump и библиотека Scapy.

1. Wireshark. Является стандартом в области графического анализа трафика. Программа обладает мощнейшим функционалом по декодированию сотен протоколов и предоставляет удобные средства фильтрации (Display Filters). Однако высокий порог вхождения и значительное потребление ресурсов памяти делают его использование не всегда оправданным для локальных задач или автоматизированных тестов. Основной минус для нашей темы — отсутствие встроенного удобного конструктора для массовой генерации специфических пакетов «из коробки».
2. Tcpdump. Легковесная консольная утилита, работающая практически во всех Unix-подобных системах. Она незаменима для быстрой диагностики на серверах без графической оболочки. Тем не менее, отсутствие визуализации и сложность анализа больших потоков данных в текстовом виде ограничивают её применение в образовательных целях и при проектировании сложных сетевых сценариев.
3. Scapy. Это мощная интерактивная библиотека на языке Python. В отличие от предыдущих инструментов, Scapy позволяет не только захватывать пакеты, но и «собирать» их вручную слой за слоем. Это дает практически безграничные возможности для генерации трафика, что и послужило основой для нашего дипломного проекта. Основной недостаток оригинальной библиотеки — отсутствие дружелюбного

графического интерфейса для конечного пользователя, что мы и планируем исправить в рамках разработки.

Для наглядного сопоставления характеристик вышеуказанных средств составим сравнительную таблицу.

Таблица 1 - Сравнительный анализ инструментальных средств анализа трафика

Критерий сравнения	Wireshark	tcpdump	Scapy	Разрабатываемое ПО
Наличие графического интерфейса (GUI)	Да	Нет	Нет (только CLI)	Да
Детальный разбор протоколов (L2-L7)	Высокий уровень	Средний уровень	Высокий уровень	Высокий уровень
Возможность генерации произвольных пакетов	Ограничено	Нет	Полная	Полная (через GUI)
Использование в автоматизированных сценариях	Сложно	Средне	Легко (Python)	Легко
Требовательность к системным ресурсам	Высокая	Низкая	Средняя	Средняя
Сохранение результатов в формате .pcap	Да	Да	Да	Да

Анализ таблицы показывает, что разрабатываемый программный комплекс должен занять нишу между сложными профессиональными анализаторами и гибкими программными библиотеками, предоставляя пользователю мощь Python/Scapy в удобной визуальной оболочке.

1.3 Постановка задачи и требования к функционалу

На основе проведенного анализа предметной области и существующих инструментов мониторинга трафика, была сформулирована задача разработки программного средства, которое обеспечит удобный интерфейс для низкоуровневой работы с сетью. Целью разработки является создание «Программного комплекса мониторинга и формирования сетевого трафика», сочетающего в себе функции пассивного слушателя (сниффера) и активного генератора.

Для достижения поставленной цели программное средство должно удовлетворять ряду функциональных требований, разделенных на три основных блока: анализ, генерация и фильтрация.

1.3.1. Требования к анализу и мониторингу трафика

Модуль анализа должен обеспечивать захват пакетов в реальном времени. В рамках реализации необходимо предусмотреть:

- возможность выбора конкретного сетевого интерфейса (Ethernet, Wi-Fi, Loopback);
- декодирование протоколов на различных уровнях модели OSI;
- визуализацию ключевых параметров пакета: IP-адреса отправителя и получателя, порты, флаги протоколов и размер полезной нагрузки (payload).

1.3.2. Требования к генерации сетевых пакетов

Ключевым отличием разрабатываемого ПО является наличие модуля формирования трафика. Пользователь должен иметь возможность:

- конструировать пакеты вручную, задавая значения заголовков;

- выбирать тип протокола (TCP, UDP, ICMP);
- задавать количество отправляемых пакетов для проведения базовых нагрузочных тестов.

1.3.3. Требования к фильтрации и хранению данных

Для работы в условиях высокой интенсивности трафика программа должна поддерживать механизмы фильтрации по протоколам или IP-адресам. Также обязательным требованием является возможность сохранения перехваченных сессий в лог-файлы или стандартные форматы (например, .txt или .csv) для последующего изучения.

Для наглядности представим схему информационных потоков программного средства.

Таблица 2 - Входные и выходные данные программного комплекса

Тип данных	Описание	Формат/Источник
Входные данные	Сетевой трафик, параметры фильтрации, параметры создаваемых пакетов	Сетевой интерфейс, ввод пользователя в GUI
Промежуточные данные	Декодированные объекты пакетов, статистика сессии	Переменные в памяти (объекты Scapy)
Выходные данные	Список перехваченных пакетов, логи активности, отчеты о генерации	Экран формы (GUI), файлы .pcap / .txt

Таким образом, постановка задачи подразумевает создание гибкого инструмента, ориентированного на практическое применение в задачах сетевого администрирования. Четкое разграничение функциональных блоков позволяет перейти к проектированию архитектуры системы, которая обеспечит выполнение всех заявленных требований.

1.4 Требования к безопасности и этике использования

Разработка и эксплуатация программных средств, обладающих функционалом генерации и модификации сетевого трафика, накладывает на разработчика и пользователя особую ответственность. В рамках данного дипломного проекта программный комплекс рассматривается исключительно как инструмент для легитимного мониторинга, диагностики и обучения.

1.4.1 Предотвращение несанкционированного использования (Misuse)

Для минимизации рисков использования программы во вред информационным системам, в процессе проектирования были заложены следующие принципы:

- Локализация воздействия: функционал генерации пакетов ориентирован на работу в пределах подконтрольного сегмента сети или виртуальной лаборатории.
- Идентификация: программа требует запуска с правами администратора, что в корпоративной среде ограничивает круг лиц, способных использовать инструмент, только квалифицированным персоналом.

1.4.2 Этические аспекты и предотвращение сетевых атак

Важной задачей является предотвращение использования модуля генерации для реализации атак типа «отказ в обслуживании» (DoS/DDoS) или подмены адресов (IP Spoofing) во внешних сетях. Этические нормы использования ПО подразумевают:

1. Соблюдение конфиденциальности: перехват данных должен осуществляться только в сетях, где пользователь имеет на это официальное разрешение.
2. Тестирование на отказ: генерация трафика высокой интенсивности должна проводиться только в изолированных тестовых средах, чтобы не нарушить работоспособность реальных бизнес-процессов.

1.4.3 Технические меры защиты

Для обеспечения безопасности самой системы, на которой запущено ПО, в алгоритмы обработки пакетов включены механизмы фильтрации и ограничения скорости отправки данных (Rate Limiting). Это предотвращает переполнение буферов сетевой карты и зависание операционной системы при проведении нагрузочных тестов.

1.5 Выбор средств разработки

Успешная реализация программного комплекса напрямую зависит от правильного выбора инструментальных средств. Учитывая сложность работы с сетевыми протоколами на низком уровне, были выбраны следующие технологии:

1. Язык программирования Python. Выбор обусловлен высокой скоростью разработки и наличием мощных библиотек для работы с сетью. Python позволяет сосредоточиться на логике анализа трафика, а не на управлении памятью, что критично для учебного дипломного проекта.
2. Библиотека Scapy. Является ключевым компонентом системы. В отличие от стандартных библиотек сокетов, Scapy позволяет манипулировать каждым полем в заголовке пакета. Это дает возможность «собирать» пакеты как конструктор.

3. Среда разработки PyCharm. Использование профессиональной IDE обеспечивает эффективную отладку кода, контроль версий и удобную работу с виртуальными окружениями.
4. Драйвер Npcap. Необходим для обеспечения доступа Python-скриптов к сетевому адаптеру в обход стандартного стека ОС, что позволяет перехватывать пакеты в «сыром» виде.

Таблица 3 - Обоснование выбора программного стека

Компонент	Выбранное средство	Основная причина выбора
Язык	Python 3.10+	Лаконичность кода, кроссплатформенность
Библиотека сети	Scapy	Низкоуровневая работа с Raw Sockets
Библиотека GUI	PyQt / Tkinter	Создание современного интерфейса
Среда (IDE)	PyCharm	Мощные инструменты рефакторинга и отладки
Системное ПО	Npcap	Стандарт для захвата пакетов в Windows

2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

2.1 Архитектура приложения и модульная структура

Проектирование архитектуры программного комплекса «Программный комплекс мониторинга и формирования сетевого трафика» основывается на принципе модульности и разделения ответственности (Separation of Concerns). Это позволяет обеспечить независимую разработку и тестирование отдельных компонентов системы, а также упрощает последующее сопровождение кода.

Выбранный стек технологий (Python + Scapy) диктует событийно-ориентированную модель архитектуры. Основным вычислительным ядром системы является движок Scapy, который взаимодействует с сетевым драйвером Npcap. Архитектуру приложения можно разделить на три логических уровня:

1. Уровень взаимодействия с операционной системой (низкоуровневый): включает в себя работу с сырыми сокетами и сетевыми интерфейсами. На этом уровне происходит захват кадров Ethernet и отправка сформированных пакетов в канал связи.
2. Уровень бизнес-логики (ядро системы): здесь сосредоточены основные алгоритмы деинкапсуляции и формирования трафика. Уровень состоит из независимых модулей:

- Модуль Sniffing (захват): отвечает за фильтрацию и первичную обработку поступающих данных.
- Модуль Packet Construction (генерация): предоставляет интерфейс для послойной сборки пакетов.
- Модуль Analytics: занимается расчетом статистики и разбором заголовков протоколов.

3. Уровень представления (интерфейсный): графическая оболочка, реализованная в среде PyCharm, которая визуализирует данные из ядра системы и принимает команды от пользователя.

Для наглядности представим модульную структуру в виде таблицы.

Таблица 4 - Описание основных модулей программного комплекса

Наименование модуля	Назначение	Используемые библиотеки/методы
Core Engine	Управление жизненным циклом приложения и потоками	Python threading / multiprocessing
Capture Module	Перехват трафика в реальном времени	Scapy (sniff() function)
Analysis Module	Разбор структуры пакетов (L2-L4)	Scapy (Layers / Fields)
Generator Module	Формирование и инъекция пакетов	Scapy (send() / sendp() functions)
GUI Wrapper	Взаимодействие с пользователем	PyQt или Tkinter
Logger Module	Запись результатов в файлы .txt / .csv	Стандартные библиотеки Python (csv, io)

Использование такой структуры позволяет минимизировать зависимости между интерфейсом и сетевой логикой. Например, при необходимости смены графической библиотеки (GUI), логика захвата и генерации трафика останется неизменной. Взаимодействие между модулями осуществляется посредством передачи объектов пакетов, что соответствует объектно-ориентированному подходу, заложенному в основу проекта.

2.2 Проектирование классовой модели

Объектно-ориентированное проектирование (ООП) является фундаментом для разработки расширяемых сетевых приложений. Классовая

модель программного средства описывает структуру системы, определяя основные сущности, их атрибуты, методы и характер взаимоотношений между ними.

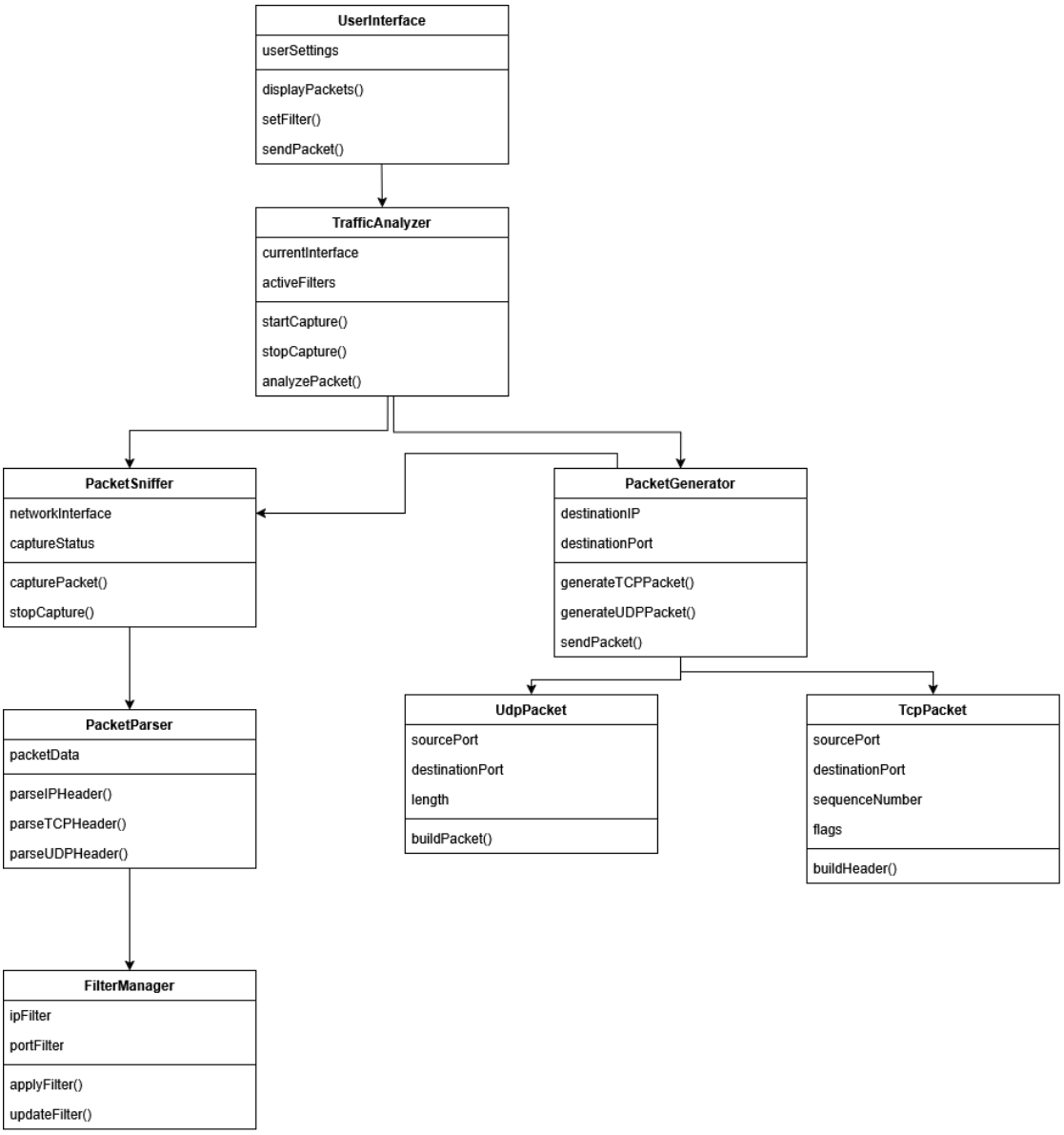

Рисунок 1 - Диаграмма классов программного средства

Согласно разработанной диаграмме, архитектура системы строится на базе трех ключевых классов, каждый из которых отвечает за свой участок работы с сетевым трафиком:

1. Класс TrafficSniffer

Выполняет роль основного модуля перехвата данных. Он взаимодействует с сетевым адаптером и управляет процессом прослушивания канала.

Атрибуты: interface (выбранный сетевой адаптер), is_running (флаг состояния захвата).

Методы:

- sniff_packets(): инициализирует функцию захвата;
- stop_sniffing(): принудительно останавливает процесс мониторинга;
- packet_callback(): функция обратного вызова, которая срабатывает при поступлении каждого нового пакета.

2. Класс PacketParser

Отвечает за интеллектуальный разбор «сырых» данных, полученных от снайффера. Этот класс преобразует набор байтов в структурированную информацию.

Атрибуты: summary (краткое описание), details (полный разбор по слоям), protocol (тип протокола: TCP, UDP и др.).

Методы:

- parse_ethernet_layer(), parse_ip_layer(), parse_transport_layer(): методы для последовательного разбора уровней модели OSI.

3. Класс TrafficGenerator

Класс, реализующий активную часть программы - формирование и отправку кастомных пакетов.

Атрибуты: target_ip, payload, packet_count.

Методы:

- build_packet(): конструктор, собирающий пакет Scapy на основе введенных пользователем данных;
- send_packet(): метод непосредственной передачи сформированного пакета в сеть.

Для наглядности представим спецификацию методов классов в таблице.

Таблица 5 - Описание методов классов на основе UML-модели

Класс	Метод	Функциональное назначение
TrafficSniffer	packet_callback()	Передача захваченного пакета в парсер для дальнейшей обработки
PacketParser	parse_ip_layer()	Извлечение IP-адресов отправителя и получателя из заголовка
TrafficGenerator	build_packet()	Формирование структуры пакета с использованием библиотеки Scapy

2.3 Моделирование процессов обработки пакетов

Для описания динамики взаимодействия объектов в системе и определения очередности обмена сообщениями между ними была разработана диаграмма последовательности (Sequence Diagram). Она позволяет визуализировать жизненный цикл сетевого пакета: от момента инициализации захвата пользователем до отображения обработанных данных на экране.

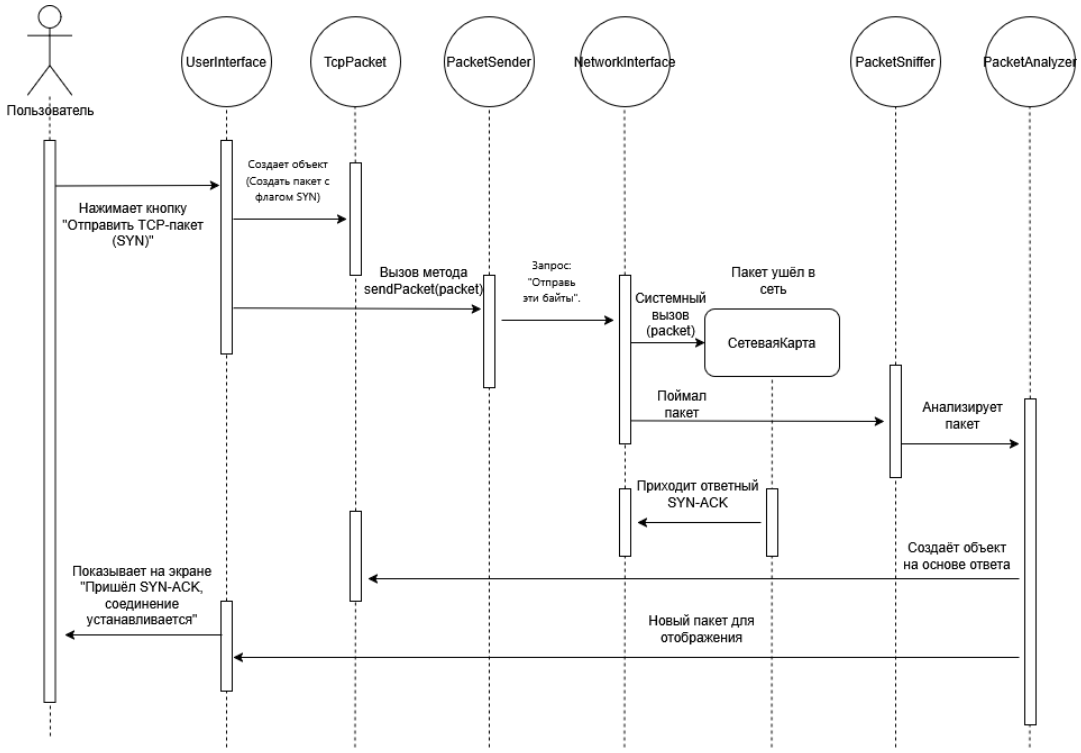

Рисунок 2 - Диаграмма последовательности процесса захвата и анализа трафика

На представленной диаграмме выделены основные участники процесса:

1. User (Пользователь): инициирует действия через графическую оболочку.
2. TrafficGUI (Интерфейс): принимает команды и отображает результаты.
3. TrafficSniffer (Захват): работает с сетевым уровнем и библиотекой Scapy.
4. PacketParser (Анализатор): выполняет логическую обработку данных.

Описание основного сценария взаимодействия:

- Инициализация: Пользователь отправляет запрос start_sniffing() через интерфейс. TrafficGUI транслирует эту команду объекту TrafficSniffer, который переводит сетевой адаптер в активное состояние.
- Захват данных: При поступлении пакета из сети TrafficSniffer перехватывает его и передает «сырые» данные (raw data) в метод parse_packet() объекта PacketParser.
- Обработка: PacketParser выполняет деинкапсуляцию, поочередно разбирая заголовки Ethernet, IP и транспортного уровня (TCP/UDP). После завершения разбора объект возвращает структурированный результат обратно в интерфейс.
- Визуализация: TrafficGUI вызывает метод display_packet(), обновляя таблицу пакетов в реальном времени, после чего система переходит в режим ожидания следующего пакета.

Такая модель взаимодействия обеспечивает высокую отзывчивость интерфейса, так как процессы захвата и отображения разделены, что предотвращает «зависание» программы при интенсивном входящем трафике.

2.4 Алгоритмы генерации и анализа трафика

Для визуализации логики работы программного комплекса и описания последовательности выполнения технологических операций была разработана диаграмма деятельности (Activity Diagram). Она позволяет

проследить пути выполнения программы в зависимости от выбора пользователя и условий среды.

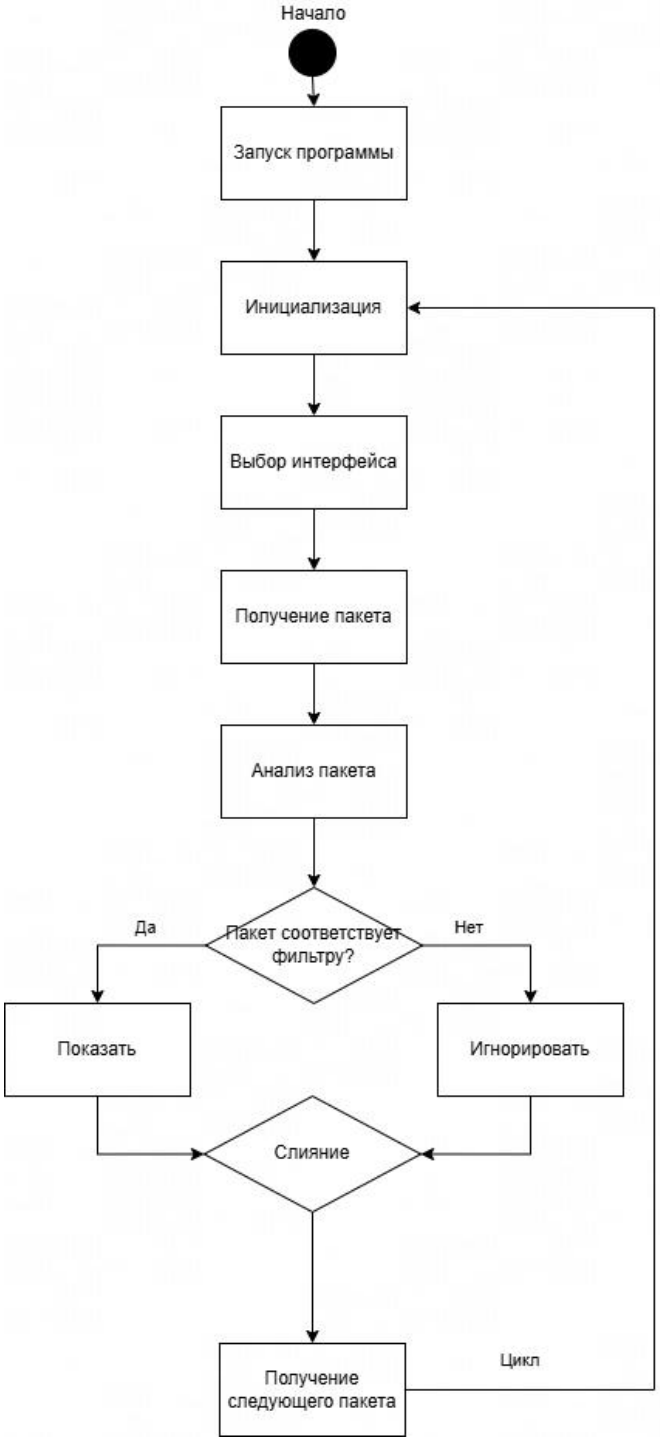

Рисунок 3 - Алгоритм работы модулей захвата и генерации пакетов

Согласно представленной схеме, процесс работы программы можно разделить на несколько ключевых этапов:

1. Инициализация и выбор интерфейса: После запуска программы («Начало») первым обязательным шагом является получение списка

доступных сетевых адаптеров. Пользователь осуществляет выбор интерфейса, через который будет производиться работа с трафиком.

2. Запуск процесса: После выбора интерфейса система переходит к этапу захвата пакетов («Запуск перехвата»).
3. Цикл обработки данных: Это центральная часть алгоритма, которая работает до тех пор, пока пользователь не решит остановить процесс.

   На каждом шаге цикла выполняются следующие действия:

- Перехват пакета: Получение «сырых» данных из сети.
- Анализ данных: Передача пакета в парсер для разбора заголовков.
- Вывод на экран: Отображение информации о пакете в таблице интерфейса.

4. Условие завершения: Алгоритм содержит логическое разветвление (ромб). Если пользователь дает команду на остановку, цикл прерывается.
5. Финализация: После остановки перехвата система предлагает (или выполняет автоматически) сохранение накопленных данных в лог-файл (форматы .csv или .txt), после чего работа программы завершается.

Разработанный алгоритм обеспечивает предсказуемость работы ПО и исключает потерю данных на этапе завершения сессии, так как сохранение логов вынесено в финальную обязательную стадию.

2.5 Структура данных и хранение результатов

В процессе работы программного комплекса возникает необходимость временного хранения больших объемов структурированной информации. В качестве основной структуры данных в памяти используется список объектов (List of Objects), где каждый элемент является экземпляром класса PacketParser.

Хранение результатов реализовано двумя способами:

1. Оперативное хранение: Использование структуры «очередь» (Queue) для передачи данных между сетевым потоком и потоком графического интерфейса. Это гарантирует, что ни один пакет не будет потерян при интенсивной нагрузке.
2. Долговременное хранение: Согласно техническому заданию, в программе реализована функция экспорта захваченного трафика.

- Формат .CSV: Используется для последующего анализа статистики в табличных редакторах (MS Excel, LibreOffice Calc). В файл записываются поля: Время, Источник, Назначение, Протокол, Длина.
- Формат .TXT: Предназначен для создания простых текстовых отчетов о сетевой активности.

2.6 Проектирование интерфейса взаимодействия

Проектирование пользовательского интерфейса (UI) программного средства выполнено с учетом требований эргономики и специфики задач сетевого администрирования. Основная цель — обеспечить быстрый доступ к мониторингу и минимизировать время на формирование сетевых запросов.

Концепция интерфейса строится на принципах «единого рабочего пространства» (Dashboard), что позволяет администратору наблюдать за потоком пакетов и одновременно управлять параметрами генерации без переключения между окнами.

Основные функциональные зоны (концептуальная структура):

1. Верхняя панель (Control Bar):

- Элементы управления: выпадающий список выбора сетевого интерфейса (Ethernet/Wi-Fi), кнопка запуска/остановки захвата («Start/Stop»), и область задания BPF-фильтров.
- Обоснование: Расположение элементов управления сверху соответствует стандартам Windows-приложений и обеспечивает удобство доступа при навигации.

2. Центральная рабочая область (Data View):

- Основная таблица отображения пакетов в реальном времени. Включает колонки: №, Время, IP-источник, IP-назначение, Протокол, Размер.
- Обоснование: Сетчатая структура позволяет быстро сканировать визуально большие объемы данных, что критически важно при анализе «шумного» трафика.

3. Нижняя панель (Inspector & Generator):

- Блок детального разбора (Inspector): при клике на пакет в таблице, в нижней части отображается подробная расшифровка заголовков (Layer 2-4).
- Блок генерации: поля ввода для задания параметров нового пакета (IP-адрес, тип протокола, флаги).
- Обоснование: Разделение окна на «общий мониторинг» и «детальный просмотр» позволяет не терять фокус при работе с конкретным сетевым событием.

4. Статусная строка (Status Bar):

- Отображение текущего состояния программы (Idle, Sniffing, Generating), количество перехваченных пакетов и индикация ошибок драйвера Npcap.

Таблица 6 - Эргономические требования к интерфейсу

Элемент UI	Целевая задача	Эргономический принцип
Таблица пакетов	Мониторинг потока	Читаемость данных (сетка)
Кнопки управления	Запуск/Остановка	Цветовая индикация (Green/Red)
Поля ввода	Генерация пакетов	Валидация типов данных (Input mask)
Разделители (Splitters)	Настройка пространства	Гибкость (масштабируемость окна)

Разработанная концепция интерфейса обеспечивает соответствие всем функциональным требованиям, заложенным в разделе 1.3, и создает базу для реализации программного кода в среде PyCharm с использованием библиотек графического вывода.

3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ

3.1 Программная реализация классов и методов

3.2 Работа с сетевыми сокетами и сырыми пакетами

3.3 Реализация механизмов фильтрации и статистики

3.4 Тестирование функциональности

3.5 Оценка производительности и нагрузки на систему

4. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЕ А

бюджетное профессиональное образовательное учреждение

Вологодской области

«Череповецкий лесомеханический техникум им. В.П. Чкалова»

Специальность 09.02.07 Информационные системы и программирование

УТВЕРЖДАЮ:

ДОЛЖНОСТЬ ПРЕПОДАВАТЕЛЯ

Должность

ФИО ПРЕПОДАВАТЕЛЯ

Ф.И.О. преподавателя

ДАТА УТВЕРЖДЕНИЯ ТЗ

дата

РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА АНАЛИЗА И ГЕНЕРАЦИИ СЕТЕВОГО ТРАФИКА

Техническое задание на дипломный проект

Листов 3

Руководитель ФИО ПРЕПОДАВАТЕЛЯ

Ф.И.О. преподавателя

Исполнитель студент ИС - 42

группа

Неклюдов Никита Сергеевич

Ф.И.О. студента

2026 год

1. ВВЕДЕНИЕ

Настоящее ТЗ распространяется на разработку программного обеспечения (ПО), предназначенного для захвата, анализа и генерации пакетов данных в сетях на базе стека TCP/IP. Программа ориентирована на использование системными администраторами и студентами ИТ-специальностей для диагностики сетей и изучения протоколов.

2. ТРЕБОВАНИЯ К ПРОГРАММЕ

2.1. Требования к функциональным характеристикам

Программа должна обеспечивать:

- Захват трафика: Перехват Ethernet-кадров и IP-пакетов с выбранного интерфейса.
- Детальный парсинг: Анализ заголовков уровней L2-L4 (MAC, IP, TCP, UDP, ICMP).
- Генерация: Формирование произвольных пакетов с заданными параметрами через библиотеку Scapy.
- Фильтрация: Выделение конкретных потоков данных по заданным критериям (IP, порт).
- Интерфейс: Графическое представление данных и диалоговое взаимодействие (согласно графу диалога).

2.2. Требования к надежности

- Устойчивость при обработке некорректных пакетов (битые заголовки).
- Валидация вводимых пользователем данных (IP-адреса, маски, порты).

2.3. Условия эксплуатации

- Запуск в ОС Windows с правами администратора.
- Наличие установленного интерпретатора Python 3.10+ и драйвера Npcap.

2.4. Требования к составу и параметрам технических средств

- ПК с ОЗУ от 4 Гб и процессором от 2 ГГц.
- Сетевая карта с поддержкой «promiscuous mode».

2.5. Требования к информационной и программной совместимости

Данный раздел определяет программную среду, в которой должна функционировать разработка:

- Операционная система: Программный комплекс должен быть совместим с семействами ОС Windows 10 и Windows 11 (архитектура x64).
- Язык программирования: Реализация должна быть выполнена на языке Python версии не ниже 3.10.
- Среда разработки: JetBrains PyCharm (версия Community или Professional).
- Системные зависимости: Для работы с сетевыми интерфейсами на низком уровне в системе должен быть установлен драйвер Npcap (в режиме совместимости с WinPcap).
- Библиотеки: Использование внешней библиотеки Scapy для формирования и декодирования пакетов.
- Права доступа: Программа требует запуска с правами Суперпользователя (Administrator) для корректного открытия raw-сокетов.

3. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

В состав документации должны входить:

- Пояснительная записка (Дипломный проект).
- Руководство пользователя.
- Листинги кода (Приложение к диплому).

4. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

Таблица 7 - Стадии и этапы разработки

Наименование этапа разработки	Сроки разработки	Результат выполнения
Разработка и утверждение ТЗ	Январь – Февраль	Утвержденное ТЗ
Проектирование архитектуры и UML-моделей	Март	Схемы классов, деятельности, последовательности
Программная реализация в среде PyCharm	Апрель – Май	Исходный код программного комплекса
Тестирование и отладка	Май	Протоколы тестирования
Оформление пояснительной записки	Июнь	Готовый дипломный проект

5. ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ

Таблица 8 - Порядок контроля и приемки

Наименование контрольного этапа	Сроки контроля	Результат выполнения
Проверка соответствия функционала ТЗ	Май	Работающая программа
Нормоконтроль документации	Июнь	Пояснительная записка по ГОСТ
Защита дипломного проекта	Июнь	Оценка ГЭК

ПРИЛОЖЕНИЕ Б

ПРИЛОЖЕНИЕ В